

Sprzętowa implementacja sieci neuronowej CNN w układzie FPGA do rozpoznawania cyfr pisanych odręcznie

Michał Szulejko

Promotor: Dr. Inż. Robert Strąkowski

12 stycznia 2026

LeNet-5 - Pierwsza sieć konwolucyjna (Yann LeCun, lata 90.)

Cztery kluczowe pomysły:

- 1 **Kernele** - małe filtry szukające wzorów
- 2 **Współdzielanie wag** - mniej pamięci
- 3 **Pooling** - zmniejszanie rozmiaru, zachowanie cech
- 4 **Warstwowe uczenie** - każda warstwa uczy się bardziej skomplikowanych wzorów

Architektura:

- Conv1: 16 filtrów, 5×5 kernel $\rightarrow$ 97K operacji
- Conv2: 32 filtry, 5×5 kernel $\rightarrow$ 558K operacji
- Dense: 10 wyników (cyfry 0-9) $\rightarrow$ 8K operacji
- Razem: 12,810 parametrów, 663K operacji

Założenia MNIST:

- Wymiary: 28×28 pikseli
- Typ: Czarno-białe
- Wartość piksela: 0-255 (czarny do białego)
- Normalizacja: Każdy piksel dzielony przez 255 $\rightarrow [0.0, 1.0]$
- Zestaw treningowy: 60,000 obrazów
- Zestaw testowy: 10,000 obrazów
- Każdy obraz ma etykietę (cyfra 0-9)

Przykład Normalizacji:

- Oryginalny piksel: 128
- Po normalizacji: $128 / 255 = 0.502$

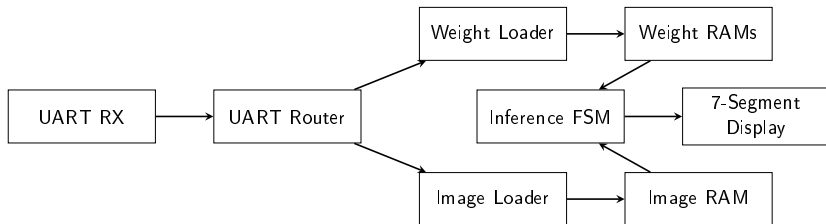
Krok 1: Trening w PyTorch

- Załaduj MNIST (60,000 obrazów treningowych)
- Trenuję sieć przez 20 epok
- Używam optymalizatora Adam
- Loss function: Cross-Entropy
- Rezultat: **99.2% dokładności** na testach

Krok 2: Post-Training Quantization (PTQ)

- Problem: Float32 wagi zajmują dużo pamięci (51 KB), obliczenia trwają dłużej oraz potrzebują więcej zasobów układu
- Rozwiązanie: Konwersja na Int8 (12.8 KB)
- Oszczędność: **4x mniej pamięci!**
- Spadek dokładności: 99.2% → **96.6%** (zaledwie 2.6%)

Inferencja - Architektura



Przepływ danych: PC → UART → FPGA

Główne moduły Verilog:

- 1 **UART Router** - odbiera dane, rozpoznaje znaczniki
- 2 **Weight Loader** - parsuje wagi (12,984 bajty)
- 3 **Image Loader** - parsuje obraz (784 bajty)
- 4 **Inference FSM** - 19 stanów: Conv1 → Pool1 → Conv2 → Pool2 → Dense

Organizacja Pamięci:

- Weight RAM: 13 KB
- Image RAM: 0.8 KB
- Intermediate Buffers: 13 KB
- Razem: ~27 KB

Połączenie Fizyczne: UART RS232

- Baudrate: 115,200 bps
- Transfer wag (13 KB)
- Transfer obrazu (784 B)

Protokół ze Znacznikami:

- Wagi: Start (0xAA 0x55) → Dane (13 KB) → End (0x55 0xAA)
- Obraz: Start (0xBB 0x66) → Dane (784 B) → End (0x66 0xBB)

Dlaczego Znaczniki?

- 1 Synchronizacja - FPGA wie, kiedy zacząć czytać dane
- 2 Bezpieczeństwo - potwierdzenie, że transfer się skończył
- 3 Odróżnienie - rozpoznanie czy to wagi czy obraz

Testowanie na MNIST:

Wersja	Dokładność	Testów
PyTorch (normalne liczby)	99.2%	10,000
Python (INT8)	96.6%	10,000
FPGA (sprzęt)	96.6%	10,000

FPGA i Python są BIT-DOKŁADNIE IDENTYCZNE

Implementacja na sprzęcie daje zaledwie 2.6% spadku z FP32

- **Przygotowanie danych** - stworzenie referencyjnych logitów za pomocą Pythona
- **Testy jednostkowe** - izolowane testowanie poszczególnych warstw
- **Test integracyjny** - pełny test porównania predykcji i logitów z uwzględnieniem wszystkich modułów
- **Wynik** - Zgodność logitów obliczonych na FPGA z symulacją gwarantuje poprawność implementacji inferencji

Wdrożyłem historyczną sieć **LeNet-5** na **FPGA** z:

- **96.6% dokładności** (spadek 2.6% względem FP32)
- **FPGA = Python** bit-dokładnie
- **27 KB pamięci**

Dzięki **kwantyzacji**, **optymalizacji w Verilog'u** i **rozproszonej pamięci RAM** mogę uruchamiać sieci na małych, tanich urządzeniach.

To pokazuje, że nie potrzebujemy GPU do inteligentnych systemów wbudowanych.